Computer Science XI

COMPLETE SKETCHNOTES

Bilingual Teacher's Edition - New Sindh Curriculum 2026

What is inside

1. Computer Systems - bits, gates, K-maps, SDLC, OSI / TCP-IP
2. Computational Thinking & Algorithms - sort, search, evaluate
3. Programming Fundamentals - Python types, operators, control, libs
4. Data and Analysis - DBMS, keys, ER, Access queries, charts
5. Application & Impacts - AI, IoT, analytics, sources, assistive tech
6. Digital Literacy - data types, collection, inquiry project, artefacts

- Source followed: CS XI lecture notes (Ch 1-6). * = highly critical for examinations.
- Every definition, comparison table, algorithm, worked example, case study and career note is kept.
- Urdu in the source is a parallel teacher track; these sketchnotes lock the English exam content.

Map of the book

Use this as a visual index. * marks golden / exam-critical clusters.

Chapter 1 - Computer Systems

Discrete vs continuous * Analog/digital - Boolean + truth tables - AND OR NOT * NAND NOR XOR XNOR - Minterms / K-maps * - LogiSim - SDLC * Waterfall vs Agile * - OSI 7 + TCP/IP *

Chapter 2 - Computational Thinking & Algorithms

CT + algorithm vs pseudocode * - Decompose / pattern / abstract - Bubble * Selection * - Linear + Binary search * - Correctness / efficiency / choose

Chapter 3 - Programming Fundamentals

Language levels + Python careers - IDE / VS Code - Variables + types * - I/O + operators * bitwise - if / loops * - Libraries + debugging

Chapter 4 - Data and Analysis

Data vs info + DBMS - Keys * ER * referential integrity * - Library case study - Access tables / forms / queries * - Charts

Chapter 5 - Application & Impacts of Computing

AI * IoT * analytics - Smart city / farm / health - Primary / secondary / tertiary * - Field impacts - Assistive tech *

Chapter 6 - Digital Literacy

Qual vs quant * - Interviews surveys prototypes obs sim * - Primary vs secondary * - Digital tools - Inquiry project * 12 steps

CHAPTER 1

COMPUTER SYSTEMS

Digital machines speak only 0 and 1. This chapter builds from quantities and signals, through Boolean algebra and gates, to software process models and the 7-layer way data travels a network.

You will sketch

1. Tell discrete (countable) from continuous (any value in a range).
2. Draw analog sine vs digital square and state reliability.
3. Build truth tables; draw AND OR NOT NAND NOR XOR XNOR.
4. Write SOP minterms and simplify with K-maps (Gray code, wrap-around).
5. Name the 6 SDLC phases and contrast Waterfall vs Agile.
6. Recite OSI 7 layers + pizza mnemonic and map them onto TCP/IP 4.

1.1.1 Discrete and Continuous Quantities

Discrete quantities are separate, countable items with specific values. You count them one by one.

Continuous quantities can take any value within a range. They flow smoothly without breaks.

Discrete - countable

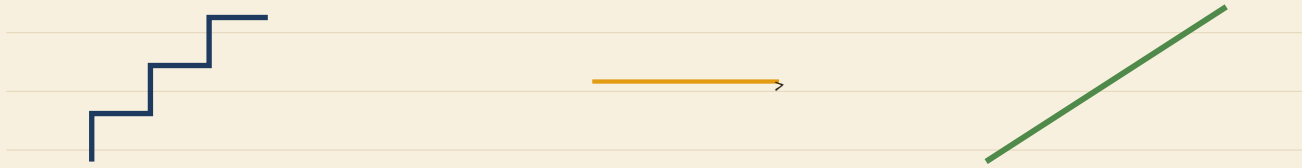
Number of students in a classroom. Number of cars in a parking lot. Each value is a distinct step.

Continuous - any point in a range

Temperature of water. Speed of a moving car. Between 20 °C and 21 °C there are infinitely many values.

~ **Stairs vs ramp**

Stairs = discrete data (fixed steps). A ramp = continuous data (you can stand on any point).



Discrete = stairs (fixed steps)

Continuous = ramp (any point)

1.1.2 Introduction to Digital Systems

A digital system is an electronic system that uses only two values: 0 and 1. These two values are called binary digits or bits.

0 means

OFF - LOW - FALSE

1 means

ON - HIGH - TRUE

Why use digital systems?

Advantage	English detail
Reliable	Less affected by noise and interference.
Accurate	No loss of quality when copying data.
Easy to design	Only two states (0 and 1) to handle.
Programmable	Can be easily controlled by software.

Examples of digital systems

- Computers: process data as 0s and 1s.
- Smartphones: store photos and music as binary data.
- Digital cameras: capture images as binary code.

1.1.3 Analog and Digital Signals

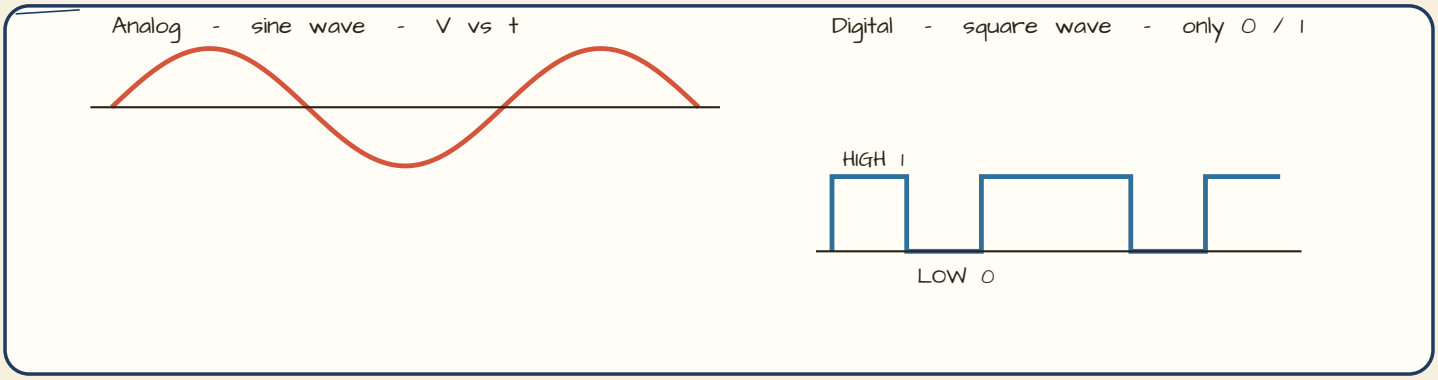
* GOLDEN TOPIC

Analog signals

Continuous signals that change smoothly over time, like a sine wave. Examples: human voice, traditional thermometers.

Digital signals

Only two levels: HIGH (1) and LOW (0). They jump instantly between values, creating a square wave. Example: computer data.



Feature	Analog data	Digital data
Nature	Continuous flow.	Discrete, separate values (bits).
Reliability	Low (affected by noise).	High (resistant to noise).

1.1.4 Boolean Algebra

Boolean algebra (developed by George Boole) deals with only two values: TRUE (1) and FALSE (0). It is the foundation of digital electronics and computer programming.

Concept	Description	Example
Variables	Letters that hold values 0 or 1.	A, B, C, X, Y
Operations	Actions performed on variables.	AND, OR, NOT
Expressions	Combinations of variables & operations.	A AND B, Y = A + B

1.1.5 Boolean Operations

1. AND (·)

Output 1 ONLY when ALL inputs are 1. Y = A · B. Real life: a door with two locks - it opens only when BOTH locks are unlocked.

2. OR (+)

Output 1 when ANY input is 1. Y = A + B. Real life: a room with two doors - you can enter if EITHER door is open.

3. NOT (' or ¯)

Inverts the input. Unary (one operand). If input is 1, output is 0. Y = A' or Y = Ā. Real life: a light switch - ON > room lit; OFF > dark.

1.1.6 Truth Tables

A truth table shows all possible input combinations and their corresponding outputs for a logic gate or circuit.

1. Count the number of inputs (n).
2. Calculate total rows = 2n.
3. List all input combinations in binary order.
4. Calculate output for each combination.

2-input n=2 rows=4

00 - 01 - 10 - 11

3-input n=3 rows=8

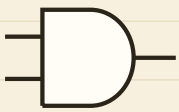
000 001 010 011 100 101 110 111

A	B	C
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

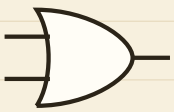
1.1.7 Logic Gates - Basic

* GOLDEN TOPIC

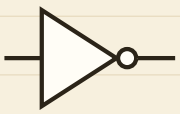
Logic gates are electronic circuits that perform Boolean operations. They are the building blocks of all digital systems.



AND Y=A·B



OR Y=A+B

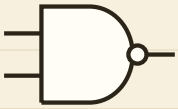


NOT Y=A'

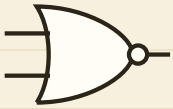
Gate	Expression	A B > Y
AND	Y = A - B	00>0 01>0 10>0 11>1
OR	Y = A + B	00>0 01>1 10>1 11>1
NOT	Y = A'	0>1 1>0

Universal gates - NAND & NOR

> * Teacher's tip
> NAND and NOR are Universal Gates because ANY logic circuit in the world can be built using only these gates.



NAND Y=(A-B)' universal



NOR Y=(A+B)' universal

Gate	Meaning	Expression	A B > Y
NAND	NOT AND	Y = (A - B)'	00>1 01>1 10>1 11>0
NOR	NOT OR	Y = (A + B)'	00>1 01>0 10>0 11>0

Advanced gates - XOR & XNOR

Gate	Expression	A B > Y	Memory
XOR	Y = A xor B	00>0 01>1 10>1 11>0	1 when inputs are eXclusively different
XNOR	Y = (A xor B)'	00>1 01>0 10>0 11>1	1 when inputs are the SAME

> * Memory trick
> XOR outputs 1 when inputs are "eXclusively" different. XNOR outputs 1 when inputs are the SAME.

1.1.8 & 1.1.10 Expressions & Minterms

Order of precedence: 1. Parentheses ( ) 2. NOT (') 3. AND (-) 4. OR (+)

Minterms - SOP Sum of Products

A product (AND) of all variables that output a 1. Complemented if the value is 0 (A'), uncomplemented if 1 (A).

Maxterms - POS Product of Sums

A sum (OR) of all variables that output a 0. Complemented if the value is 1 (A'), uncomplemented if 0 (A).

A	B	Y	Minterm (SOP)	Maxterm (POS)
0	0	0	-	A + B
0	1	1	A'B	-
1	0	0	-	A' + B
1	1	1	AB	-

i Final SOP from the 1-rows

Y = A'B + AB

1.1.11 Logic Diagrams

* GOLDEN TOPIC

A logic diagram shows how logic gates are connected to create a circuit. Based on precedence, the operation with highest priority is drawn first.

*** Example: Y = A - B + C**

AND (-) has higher precedence than OR (+). Draw the AND gate first (A and B), then OR that result with C to get Y.

A → AND with B → OR with C → Y

1.1.13 Karnaugh Maps (K-Maps)

* GOLDEN TOPIC

A K-map is a graphical tool used to simplify Boolean expressions. Adjacent cells differ by only ONE bit (Gray code). Group 1s in powers of 2 (1, 2, 4, 8) to simplify equations without complex algebra.

Two-variable K-map (A, B) - 22 = 4 cells

A\B	B=0	B=1
A=0	0 A'B'	1 A'B
A=1	2 AB'	3 AB

2-var K-map: 4 cells. Adjacent cells differ by ONE bit (Gray code).

*** Grouping rule**

Make as large groups as possible (size 1, 2, 4, 8). Overlapping is allowed. Variables that change state (0>1) inside a group are eliminated.

Three-variable K-map (A, B, C) - 23 = 8 cells

Columns follow Gray code: 00, 01, 11, 10 so adjacent cells differ by only one variable.

A\BC	00	01	11	10
A=0	m0 A'B'C'	m1 A'B'C	m3 A'BC	m2 A'BC'
A=1	m4 AB'C'	m5 AB'C	m7 ABC	m6 ABC'

Gray columns 00-01-11-10. Edges WRAP: far-left groups with far-right.

> **! Wrap-around is allowed**
> A cell on the far left can be grouped with a cell on the far right. Edges are neighbours.

LogiSim Evolution - simulation tool

LogiSim Evolution is a graphical tool for designing and simulating logic circuits. Students visually build the truth tables and Boolean expressions they learn in theory.

Interface areas

Area	What it does
Menu bar	File, Edit, Project, Simulate.
Toolbar	Wiring, Poke, AND, OR, NOT gates.
Explorer (left)	Component libraries: Gates, I/O, Wiring.
Attribute table (bottom left)	Number of inputs, facing direction.
Drawing canvas (right)	Dotted grid where you place and wire gates.

Wiring colours during simulation

Bright green
Logic 1 (HIGH)

Dark green
Logic 0 (LOW)

Blue
Unknown / unconnected pin

Red
Error / short circuit

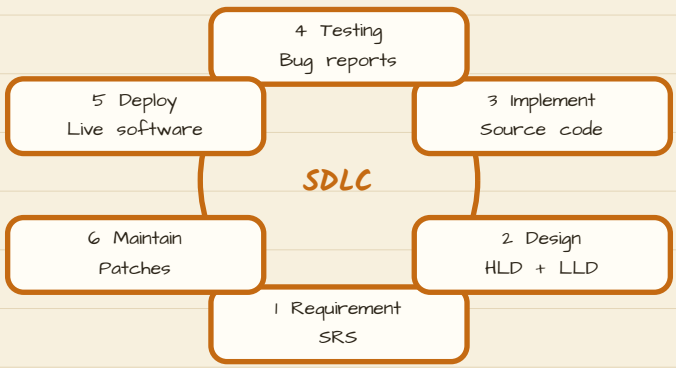
Build Y = A · B

1. Select the AND gate from the toolbar or Gates library; place it on the canvas.
2. Place two Input Pins (square icon with a 0) to the left; label them A and B.
3. Place an Output Pin (circle icon with a 0) to the right; label it Y.
4. Wiring tool: drag wires from inputs to AND ports. Wires must be horizontal or vertical.
5. Wire the gate output to pin Y.
6. Poke tool (pointing finger) toggles A and B between 0 and 1 to test. Bright green = Logic 1.

1.2 Software Development Life Cycle

`* GOLDEN TOPIC`

SDLC is a structured process that guides software development from beginning to end. It ensures clear goals, better planning, risk reduction, and high quality.



Phases 1-3

Phase	Activities	Deliverable
1. Requirement analysis	Meet clients, understand needs, list features. Most important phase.	SRS - Software Requirements Specification
2. System design	High-Level Design (architecture), Low-Level Design (database, UI), flowcharts.	Design document
3. Implementation	Write actual code (Python, Java, .) from the design docs.	Working source code

~ *Building a house*

Phase 1 = ask the family what rooms they need. Phase 2 = architect draws blueprints. Phase 3 = builders lay bricks.

Phases 4-6

Phase	Activities	Deliverable
4. Testing	Find and fix bugs (Black Box vs White Box).	Test reports, bug-free software
5. Deployment	Release to end-users: Direct, Phased, or Pilot.	Live software
6. Maintenance	Support, updates, fix post-release issues.	Patches, updated versions

Black Box testing

- Tester does NOT see code.
- Tests functionality (inputs / outputs).
- Done by dedicated testers.

vs

White Box testing

- Tester CAN see the code.
- Tests internal logic / structure.
- Done by developers.

1.2.4 The Waterfall Model

* GOLDEN TOPIC

A linear, sequential approach. Each phase must be completed before the next begins. There is no going back to a previous phase easily.

When to use?

- Requirements are clear, fixed, and well-understood.
- Project is small to medium size.
- Technology is stable.

Advantages (pros)

- Simple to understand and manage.
- Well-documented at every step.

vs

Disadvantages (cons)

- Not flexible to changes.
- Testing happens very late in the process.

* *Case study - Al-Noor Library Management System*

Used as a Waterfall case because requirements were 100% fixed before coding began.

1.2.4 The Agile Model

* GOLDEN TOPIC

An iterative approach. Software is developed in small cycles called sprints (usually 2-4 weeks). Customer feedback is gathered continuously.

When to use?

- Requirements are unclear or may change.
- Customer wants frequent progress updates (early delivery).
- Complex or innovative projects.

Advantages (pros)

- Highly flexible to changes.
- Early delivery of working software.

vs

Disadvantages (cons)

- Less documentation.
- Unpredictable end-time and cost.

*** Case study - Al-Noor Student Mobile Application**

Used Agile because teachers, parents, and students constantly requested new features during development.

Waterfall vs Agile - full comparison

Dimension	Waterfall	Agile
Flexibility	Rigid. Hard to change later.	Very flexible. Next sprint absorbs change.
Customer involvement	Only at start (requirements) and end (delivery).	Continuous - after every sprint.
Delivery	Single delivery at the very end.	Frequent partial deliveries after every sprint.
Testing	Late, in a dedicated testing phase.	Continuous testing inside every sprint.
Documentation	Heavy documentation required.	Light. Focus is on working software.

1.3 Communication Models (OSI)

* GOLDEN TOPIC

A communication model explains how data travels from one device to another through a network. The OSI (Open Systems Interconnection) Model was developed by ISO in 1984. It divides communication into 7 layers.

*** Pizza mnemonic (bottom > top)**

Please Do Not Throw Sausage Pizza Away = Physical, Data Link, Network, Transport, Session, Presentation, Application.

7	Application	HTTP HTTPS FTP SMTP DNS - services for the user
6	Presentation	SSL/TLS JPEG MP3 ASCII - translate, encrypt, compress
5	Session	NetBIOS RPC - open / maintain / close sessions
4	Transport	TCP reliable - UDP fast - ports + end-to-end
3	Network	IP ICMP routers - logical address + routing
2	Data Link	Ethernet MAC switches - frames + MAC address
1	Physical	Cables hubs Wi-Fi radio - raw bits 0/1

* TCP vs UDP at the Transport layer

Use TCP when accuracy matters (web browsing, email). Use UDP when speed matters more than perfection (video streaming, online gaming).

TCP/IP model vs OSI model

The TCP/IP model is the practical model used on the real Internet. It condenses the 7 OSI layers into just 4 layers.

TCP/IP (4 layers)	Corresponds to OSI
1. Application	Application + Presentation + Session (7, 6, 5)
2. Transport	Transport (4)
3. Internet	Network (3)
4. Network Access	Data Link + Physical (2, 1)

Chapter wrap - keep these

- Discrete = countable steps; continuous = any value on a range (stairs vs ramp).
- Digital systems use bits 0/1; digital signals resist noise better than analog.
- AND all-1s, OR any-1, NOT flips; NAND/NOR are universal; XOR = different, XNOR = same.
- Precedence: ( ) then NOT then AND then OR. SOP from minterms of 1-rows.
- K-map: Gray code, groups of 1/2/4/8, wrap edges, eliminate changing variables.
- SDLC 6 phases. Waterfall = linear fixed; Agile = sprints + continuous feedback.
- OSI 7 layers (pizza mnemonic). TCP/IP 4 layers map onto them. TCP = accurate, UDP = fast.

CHAPTER 2

Computational Thinking & Algorithms

Think like a computer scientist first, then write the steps. This chapter is the exam spine for algorithm vs pseudocode, the three CT strategies, two sorts, two searches, and how you judge an algorithm.

You will sketch

1. Define CT and list why algorithms matter.
2. Contrast algorithm vs pseudocode (recipe vs kitchen manual).
3. Decompose, find patterns, abstract noise.
4. Dry-run Bubble Sort and Selection Sort on {8,4,1,9,3}.
5. Linear search on unsorted data; Binary search only on sorted data.
6. Judge correctness, efficiency, clarity - then pick the tool.

2.1 Computational Thinking

Computational Thinking (CT) is a systematic approach to solving problems. It involves breaking problems down, identifying patterns, abstracting details, and developing algorithms.

> **i** *Did you know?*
> Pseudocode is a human-readable representation of an algorithm using plain English and structured programming keywords. It is the bridge between human logic and machine code.

2.2 Algorithm

* GOLDEN TOPIC

An algorithm is a step-by-step logical procedure used to solve a problem. It is conceptual and completely independent of any programming language.

2.2.1 Importance of algorithms

- Structured approach to problem-solving - complex tasks become manageable.
- Optimize time and memory - crucial for large-scale applications.
- Reusable for similar problems - saves time and effort.
- Accuracy and consistency - reduce human error.
- Blueprint before actual code implementation.

2.2.2 Algorithm vs Pseudocode

* GOLDEN TOPIC

Both are used in the design phase, but they serve different purposes.

Point	Algorithm	Pseudocode
What it is	Step-by-step logical procedure to solve a problem.	Human-readable representation using plain English & keywords.
Nature	Conceptual and language-independent.	Semi-formal and structured but not executable code.

Point	Algorithm	Pseudocode
Purpose	Define what needs to be done logically.	Present how it can be implemented in a readable format.
Form	Plain steps, bullets, or numbered points.	Structured constructs (IF, FOR, WHILE) like programming.
Focus	Logic and steps of the problem.	Representing steps clearly before coding.
Audience	Anyone, including non-programmers.	Programmers or students preparing for coding.
Format	No fixed format; can be informal.	Follows standard pseudocode conventions.

~ *Recipe vs kitchen manual*

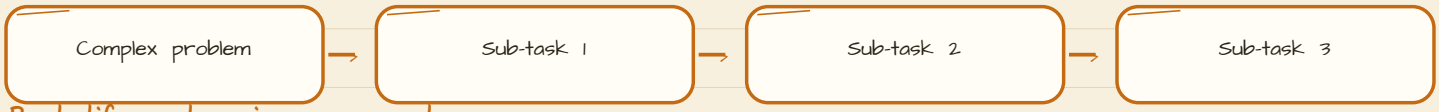
Algorithm: "Bake a cake at 350 degrees." Pseudocode: SET temperature TO 350; WHILE cake is not baked, WAIT.

2.3 Algorithmic Strategies

2.3.1 Decomposition

* GOLDEN TOPIC

A complex task or system is broken into smaller, manageable parts. By dividing a big problem into simpler steps, it becomes easier to understand, develop, and solve.



Real life - learning a new language

Subtask	Description
Learn vocabulary	Character sets for people, actions, things (I, eat, apple).
Grammar rules	Structure: Subject > Verb > Object.
Learn tenses	Present, past, future (I eat vs I ate).
Sentence building	Combine words using rules to create valid sentences.

- Reduces complexity - focus on one piece at a time.
- Helps debugging - find exactly where the mistake is (verb tense or word order).
- Promotes reusability - a small learned part applies elsewhere.

2.3.2 Pattern Recognition

Identifying regularities, repetitions, or rules in data. It reduces algorithm complexity and is the foundation of loops.

Ask yourself

What is repeating? What changes in a predictable way? Can I generalize this?

Triangle of stars

Each row adds 1 star to the previous row. Row 1 = 1 star, Row N = N stars. Write one generic loop instead of 5 separate print statements.

2.3.3 Abstraction

Hide unnecessary features and focus only on the essential ones. Abstraction reduces complexity by ignoring noise.

Making tea - apply abstraction

1. Necessary (core logic)	2. Situation variables	3. Ignore / noise
1 Boil water	Ingredients: black, green, milk	Brand of kettle
2 Add tea leaves/bag	Additions: sugar, lemon	Cup colour or design

1. Necessary (core logic)	2. Situation variables	3. Ignore / noise
3 Steep a few minutes	Tools: stove vs kettle	Where tea is made (kitchen, office)
4 Pour into cup		
5 Serve		

> **i Core abstracted logic**
> Boil > Add tea > Steep > Pour > Serve. Independent of cup colour or kettle brand.

2.4 Sorting and Searching

Both are fundamental. They work on lists of the same data type.

Sorting

Arranging a collection. Ascending (0-9, A-Z) or Descending (9-0, Z-A). Used to organise data.

Searching

Locating a specific element within a list to find its position or existence.

2.4.1 Bubble Sort

* GOLDEN TOPIC

Elementary sort: repeatedly compare adjacent elements and swap them if they are in the wrong order.

Bubble Sort - algorithm + pseudocode (same idea)

```
Start
DATA = {8,4,1,9,3}   N = 5
For outer = 0 to N-1
    For inner = 0 to N-2-outer
        IF DATA[inner] > DATA[inner+1]
            Temp = DATA[inner]
            DATA[inner] = DATA[inner+1]
            DATA[inner+1] = Temp
        End If
    End For
End For
End
```

Worked example Unsorted: 8, 4, 1, 9, 3

Pass / pair	Compare	Result
P1 [8,4],1,9,3	8 > 4 > swap	4, 8, 1, 9, 3
P1 4,[8,1],9,3	8 > 1 > swap	4, 1, 8, 9, 3
P1 4,1,[8,9],3	8 NOT > 9	no change
P1 4,1,8,[9,3]	9 > 3 > swap	4, 1, 8, 3, 9 (9 parked)
P2 [4,1],8,3,9	4 > 1 > swap	1, 4, 8, 3, 9
P2 1,[4,8],3,9	4 NOT > 8	no change
P2 1,4,[8,3],9	8 > 3 > swap	1, 4, 3, 8, 9
P3 1,[4,3],8,9	4 > 3 > swap	1, 3, 4, 8, 9
P4 [1,3],4,8,9	1 NOT > 3	sorted 1, 3, 4, 8, 9

Advantages

- Easiest to visualise and explain.
- In-place - no extra memory.
- Stable (equal items keep original order).

vs

Disadvantages

- Nested loops - time grows badly.
- Massive number of swaps.
- Useless for real-world Big Data.

2.4.1 Selection Sort

* GOLDEN TOPIC

Repeatedly select the smallest element from the unsorted portion and move it to its correct position at the front.

Selection Sort pseudocode - inner loop starts at outer+1

```
Start
DATA = {8,4,1,9,3}   N = 5
For outer = 0 to N-1
    For inner = outer+1 to N-1
        IF DATA[outer] > DATA[inner]
            swap DATA[outer], DATA[inner]
        End If
    End For
End For
End
```

Teacher's tip

Selection-sort inner loop starts at outer+1. Bubble Sort's inner loop always starts at 0.

Worked example Unsorted: 8, 4, 1, 9, 3

Step	Action	List
Pos 1	8>4 swap; 4>1 swap; 19; 13	1 locked, 8,4,9,3
Pos 2	8>4 swap; 49; 4>3 swap	1, 3 locked, 8,9,4
Pos 3	8>4 swap	1, 3, 4 locked, 9,8
Pos 4	9>8 swap	1, 3, 4, 8, 9

Advantages

- Very easy to code.
- Fewer swaps than Bubble Sort.
- No extra memory.

vs

Disadvantages

- Nested loops - exponential-feeling growth.
- Blind - finishes every loop even if already sorted.
- Not stable (may swap equal items' order).

2.4.2 Linear Search

Also called sequential search. Examine each element one by one until the target is found. Works on both sorted and unsorted data.

Linear / sequential search

```
DATA = {8, 3, 6, 1, 7, 2, 4, 5}   N = 8   FindValue = 7
ValueFound = 0
For PTR = 0 to N-1
    If DATA[PTR] == FindValue then
        ValueFound = 1
        Exit Loop
    End If
End For
If ValueFound == 1 then Print DATA[PTR] else Print "Data Not Found"
```

Step	Element	== 7 ?	Result
1	8	8 == 7 ?	Not found
2	3	3 == 7 ?	Not found
3	6	6 == 7 ?	Not found
4	1	1 == 7 ?	Not found
5	7	7 == 7 ?	Found - exit loop

Advantages

- Works on scrambled or sorted data.
- Extremely simple (one loop, one if).
- Faster for very small lists (no setup).
- No extra space.

vs

Disadvantages

- Target at the end of 1 million items > 1 million checks.
- Time grows linearly (double the list = double the time).
- Inefficient if you search the same list thousands of times.

2.4.2 Binary Search

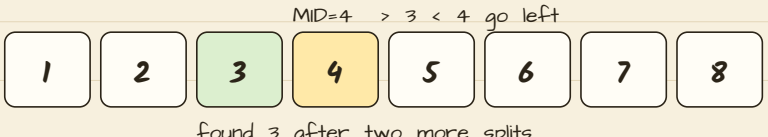
* GOLDEN TOPIC

Repeatedly divide the search interval in half. Crucial limitation: it only works on a SORTED list.

Binary search - half the interval each time

```
DATA = {1,2,3,4,5,6,7,8,9}   (must be sorted)
BEG = 1, END = 9   MID = INT((BEG+END)/2)   Item = 7   LOC = NULL
While BEG <= END AND DATA[MID] != Item
    IF Item < DATA[MID] then END = MID - 1
    Else BEG = MID + 1
    MID = INT((BEG+END)/2)
End While
If DATA[MID] == Item then LOC = MID else LOC = NULL
Print found or "Not Found"
```

Worked example Sorted [1,2,3,4,5,6,7,8] search 3



1. MID = 8/2 = 4 (value 4). 3 < 4 > go left.
2. Sub-list [1,2,3]. MID ~ 2 (value 2). 3 > 2 > go right.
3. Sub-list [3]. Value 3 == 3. FOUND.

Advantages

- Extremely fast for Big Data. ~20 steps in 1 million elements.
- Huge savings when a static list is searched repeatedly.

vs

Disadvantages

- Must be sorted first - fails on scrambled data.
- Needs random access (arrays, not easily linked lists).
- More complex - off-by-one bugs are common.

2.5 Evaluation Criteria of an Algorithm

2.5.1 Correctness

Output matches expected outcomes, handles edge cases, and is guaranteed to finish (no infinite loops).

2.5.2 Efficiency

Time (speed as input grows) and space (memory). Does it scale?

2.5.3 Clarity (readability)

Easy to read, understand, and modify. Clear steps, meaningful variables, good indentation.

Trace table - even / odd checker

Input	Operation (/2, remainder)	Remainder == 0?	Result
3	3/2 Q=1 R=1	No	Odd
6	6/2 Q=3 R=0	Yes	Even
15	15/2 Q=7 R=1	No	Odd

2.6 How to decide which algorithm to use

Ask: What is the input size? Is the data sorted? Do we need speed or memory efficiency?

Unsorted search > Linear Sorted search > Binary Need order > a sorting algorithm

Chapter wrap - keep these

- CT = Decomposition + Pattern Recognition + Abstraction + Algorithms.
- Algorithm = conceptual steps. Pseudocode = structured English before code.
- Bubble / Selection = simple nested-loop sorts; slow on large data. Selection does fewer swaps.
- Linear search = one-by-one, works unsorted. Binary search = split in half, MUST be sorted.
- Judge with Correctness, Efficiency (time/space), Clarity - then pick.

CHAPTER 3

PROGRAMMING FUNDAMENTALS

Computers are dumb - they only follow instructions. A program is that set of instructions. This chapter is Python-first: types, I/O, every operator family, control flow, libraries, and how to read a traceback.

You will sketch

1. Classify low / mid / high-level languages and name Python careers.
2. Turn VS Code into an IDE with the Microsoft Python extension.
3. Name variables legally; list int float str bool list tuple dict set.
4. print / input / f-strings / type-cast.
5. Arithmetic, assignment, relational, logical, membership, bitwise.
6. Sequence, if / elif / else / nested, for+range, while, break/continue.
7. Import styles, math/random/datetime, pip libraries, 3 error types.

3.1 Programming

Computers cannot think without instructions. A set of instructions is a program. The language used to give those instructions is a programming language.

3.1.1 Programming language types

Type	Description	Examples
Low level	Closer to the machine (binary 0s & 1s). Very hard for humans.	Machine language (binary)
Mid level	Bridges the gap. Hardware access, but mnemonics are readable.	Assembly - MOV AX, BX
High level	Easy for humans. English-like words.	C++, Java, Python - print("Hello")

C++

Powerful, closer to hardware. System software, OS, high-performance games.

Java

Write once, run anywhere. Enterprise apps, Android, servers.

Python

Simple readable syntax. Data Science, AI, web, beginners.

3.2 Career growth with Python

- Software development - desktop and web applications.
- Data science and analysis - reports and predictions.
- AI and machine learning - intelligent systems, chatbots, recommenders.
- Web development - Django and Flask.
- Automation and scripting - repetitive office or lab tasks.
- Cybersecurity and networking - security tools and network analysis.

*** Why Python?**

Students can focus on problem-solving and logic rather than memorising complex syntax.

3.3 Integrated Development Environment (IDE)

An IDE is a software tool that helps programmers write, run, and manage code in one place - a complete workspace.

- Code editor - write and edit.
- Syntax highlighting - colours for readability.
- Auto-completion - suggests code while you type.
- Run / execute tool - runs the program.
- Debugger - find and fix errors.

3.3.1 Visual Studio Code

VS Code is a free, lightweight, powerful editor. It is not a full IDE by default - install extensions to turn it into one.

Element	Description
1. Top menu bar	Files, editing, navigation, workspace (File, Edit, Selection, View, Go, Run).
2. Side bar	Explorer, Search, Source Control, Run & Debug, Extensions.
3. Explorer	Project files and folders - create, open, manage .py files.
4. Search bar	Find and replace in the open file or the whole project.
5. Source control	Git - track changes, undo, collaborate.
6. Run & Debug	Execute, pause on lines, inspect variables.
7. Extensions	App Store for VS Code: Python, Prettier, Python Debugger.

*** Demo setup**

Install the Python extension by Microsoft before class so syntax highlighting and the Run button work.

3.4 Fundamentals of Python Programming

* GOLDEN TOPIC

Python programs process data with statements and show output. They run top-to-bottom sequentially.

1. Comments (#)

Notes for humans. The interpreter ignores them completely.

2. Indentation (spaces)

C++/Java use { }. Python uses indentation - it is mandatory. Standard: 4 spaces (or 1 tab). Mixing tabs and spaces causes IndentationError.

Comments + mandatory indent

```
# This is a comment. It explains the code below.
print("Hello, World!")
if 5 > 2:
    # The space before print is the Indentation
    print("Five is greater than two!")
```

3.4.1 Variables

* GOLDEN TOPIC

A variable is a named space in computer memory used to store data - a container. You do not declare the type; Python figures it out.

- May contain letters, numbers, and underscores (_).
- MUST start with a letter or underscore, NOT a number. 1name is invalid.
- Case-sensitive: age and Age are different.
- Cannot use Python keywords (if, for, while).

Valid

- name = "Ali"
- age = 16
- my_height = 5.7
- score1 = 100

vs

Invalid

- 1name = "Ali" # starts with number
- my height = 5.7 # space
- if = 100 # keyword

Data types in Python

Name	Type	Description & example
Integers	int	Whole numbers. age = 16
Floating point	float	Decimals. height = 5.7
Strings	str	Text in quotes. name = "Sammy" or year = "2026"
Boolean	bool	True or False (capital T/F). is_student = True
Lists	list	Ordered, changeable [brackets]. marks = [85, 90, 75]
Tuples	tup	Ordered, unchangeable (parentheses). coords = (10, 20)
Dictionaries	dict	Key:value pairs {braces}. student = {"name":"Ahmed", "age":16}
Sets	set	Unordered unique objects. vowels = {'a', 'e', 'i'}

3.4.2 Input and output handling

print(), f-strings, input() + casting

```
print("Hello Class!")
name = "Ali"
print("Welcome,", name)
age = 16
print(f"My name is {name} and I am {age} years old.")

user_name = input("Enter your name: ")          # always a string
age = int(input("Enter your age: "))            # type-cast to int
height = float(input("Enter your height: "))    # type-cast to float
```

3.4.3 Operators and operands

* GOLDEN TOPIC

An operand is the value or variable. An operator is the symbol that performs the action. 10 (operand) + (operator) 20 (operand)

1. Arithmetic

Op	Purpose	Example	Result
+	Addition	5 + 3	8
-	Subtraction	10 - 4	6
*	Multiplication	4 * 2	8
/	Division (always float)	9 / 2	4.5
//	Floor division (drops decimal)	9 // 2	4
%	Modulus (remainder)	9 % 2	1
**	Exponentiation (power)	2 ** 3	8

2. Assignment

Op	Example	Equivalent to
=	x = 5	Assign 5 to x
+=	x += 2	x = x + 2
-=	x -= 1	x = x - 1
*=	x *= 3	x = x * 3

3. Relational (comparison) - return True / False (a=10, b=5)

Op	Meaning	Example	Result
==	Equal to	a == b	False
!=	Not equal	a != b	True
>	Greater than	a > b	True
<	Less than	a < b	False
>=	Greater or equal	a >= 10	True

4. Logical

and

True ONLY if both conditions are true.

or

True if at least one condition is true.

not

Reverses the result (True becomes False).

5. Membership

```
name = "Python"
print("P" in name)      # True
print("z" not in name)  # True
```

6. Bitwise - treat numbers as bits; bin() shows binary

Op	Rule	Bits	Example a=10 (1010), b=6 (0110)
& AND	1 only if BOTH bits are 1	00>0 01>0 10>0 11>1	10 & 6 = 0010 = 2
| OR	1 if AT LEAST one bit is 1	00>0 01>1 10>1 11>1	10 | 6 = 1110 = 14
^ XOR	1 if bits DIFFER	00>0 01>1 10>1 11>0	different bits survive
~ NOT	Flip every bit; also -(x+1)	0>1 1>0	~10 gives -11
<< >>	Left x2 per shift; right /2 (floor)	5<<1 > 10; 5<<2 > 20; 20>>1 > 10	Left adds 0s on the right

3.5 Control Structures

* GOLDEN TOPIC

Control structures decide the order statements run. Almost every program uses Sequence, Selection, and Repetition.

3.5.1 Sequence

Default: statements run one after another, strictly top to bottom.

```
Start > Statement 1 > Statement 2 > End
print("Enter two numbers")
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
total = a + b
print("Sum =", total)
```

3.5.2 Selection statements

Selection alters the flow based on a condition (True/False).

if

Runs a block ONLY if the condition is True; otherwise it is skipped.

```
age = int(input(...))
if age >= 18:
print("You are an Adult.")
print("Program continues...") # always
```

if-else

Two paths - one of them always runs.

```
if age >= 18: print("Apply for CNIC.")
else: print("Apply for CRC (B-Form).")
```

! Formatting rule

Colon (:) after if / else / elif. The block MUST be indented.

if-elif-else - first True wins, rest ignored

```
marks = int(input("Enter marks: "))
if marks >= 80:    print("A-1 Grade")
elif marks >= 70:  print("A Grade")
elif marks >= 60:  print("B Grade")
else:              print("Fail or below B")
```

Nested if - multiple levels of checks

```
age = 20
salary = 35000
if age >= 18:
    if salary >= 30000:
        print("Loan APPROVED!")
    else:
        print("Salary too low for loan.")
else:
    print("Too young for loan.")
```

★ 3.5.3 Repetition statements (loops)

★ GOLDEN TOPIC

A loop executes a block repeatedly so you do not write the same code over and over.

for + range() - when the count is known

Syntax	How it works	Example > output
range(stop)	0 up to but NOT including stop	range(3) > 0 1 2
range(start, stop)	start up to (not including) stop	range(2, 5) > 2 3 4
range(start, stop, step)	jump by step	range(1, 6, 2) > 1 3 5

```
for i in range(0, 6, 2):
    print("Even:", i)   # 0 2 4
```

while - when the count is NOT known

```
count = 1
while count <= 3:
    print(count)
    count += 1   # MUST update or infinite loop
# 1 2 3
```

break

- Exits the loop immediately.
- for i in range(5):
if i == 3: break
print(i)
prints 0 1 2

vs

continue

- Skips this iteration, goes to the next.
- for i in range(5):
if i == 3: continue
print(i)
0 1 2 4 (skips 3)

Nested loops

Inner loop runs completely for every single iteration of the outer loop.

```
for outer in range(1, 3):          # 1, 2
    for inner in range(1, 4):      # 1, 2, 3 each time
        print(f"Outer: {outer}, Inner: {inner}")
# (1,1)(1,2)(1,3)(2,1)(2,2)(2,3)
```

for loop

- Known number of iterations.
- Automatically updates the counter.
- Cleaner for counting tasks.

vs

while loop

- Unknown count - condition-based.
- You must update the counter (infinite-loop risk).
- Better for "keep going until user says Stop".

Activity - shopping bill (sequence + selection + repetition)

```
20% off when total >= 5000
print("===== SHOPPING BILL =====")
total = 0.0; count = 0; shopping = True
while shopping:
    item = input("Enter Item Name (or 'done' to stop): ")
    if item.lower() == "done":
        shopping = False
    else:
        price = float(input(f"Enter Price of {item}: "))
        if price <= 0: print("Invalid price!")
        else:
            total += price; count += 1
discount = total * 0.20 if total >= 5000 else 0
print(f"Total Items: {count}  Bill: {total}  Pay: {total-discount}")
```

3.6 Libraries in Python

Libraries are pre-written code with helpful functions. Use them instead of writing complex code from scratch.

Style	Code	Note
1. Entire module	import math res = math.sqrt(25)	Must prefix module.func()
2. Alias	import math as m res = m.sqrt(16)	Nickname
3. Specific function	from math import sqrt res = sqrt(36)	No prefix - best practice
4. Several functions	from math import sqrt, pi	

Style	Code	Note
5. Everything *	from math import * res = sqrt(64)	Avoid - worse for memory/clarity

3.6.1 Built-in libraries (already installed - just import)

math

math.pi - math.ceil(4.3)>5 - math.floor(4.9)>4 - math.pow(2,4)>16.0 - math.sqrt(81)>9.0

random

random.randint(1, 10) integer in range.
random.random() float in [0.0, 1.0).

datetime

now = datetime.datetime.now()
now.strftime("%d-%m-%Y") > 27-09-2026

3.6.2 Third-party - pip install library_name

Library	Primary use
NumPy	Heavy numerical computing and large arrays.
Pandas	Data analysis, table-like data (Excel/CSV).
Matplotlib	Visualisation - graphs and charts.
TensorFlow / PyTorch	Artificial intelligence and machine learning.

3.7 Debugging code in Python

Debugging is identifying, tracing, and resolving bugs so the program works correctly.

1. Syntax errors

Grammar of Python is broken. The program will NOT run until fixed.
print("Hello World" # missing)

2. Runtime errors (exceptions)

Code starts, then an impossible operation crashes it.
x = 10 / 0 > ZeroDivisionError

3. Logical errors

Runs without crashing but the answer is wrong.
area = length + width # should be *

Reading a traceback

A traceback is a roadmap: error type, file, exact line.

```
Line 1 tried to turn letters into an int > ValueError
Traceback (most recent call last):
  File "debug_program.py", line 1, in <module>
    x = int("abc")
ValueError: invalid literal for int() with base 10: 'abc'
```

Chapter wrap - keep these

- Python is high-level and readable - AI, data, web.
- VS Code + Microsoft Python extension ~ IDE.
- Variables are dynamically typed. Types: int float str bool list tuple dict set.
- print outputs; input always returns a string - cast with int()/float().
- Operators: arithmetic // % ** , assignment, relational, and/or/not, in/not in, & | ^ ~ << >>.
- Control: Sequence, Selection (if/elif/else/nested), Repetition (for/range, while, break/continue).
- Libraries: built-in math/random/datetime; third-party via pip (NumPy, Pandas, Matplotlib).
- Errors: Syntax (won't run), Runtime (crash), Logical (wrong answer). Read the traceback.

CHAPTER 4

DATA AND ANALYSIS

Raw facts become decisions only after they are organised. This chapter is the database blueprint: data vs information, keys, ER diagrams, referential integrity, the library case study, and Microsoft Access objects.

You will sketch

1. Define data, information, database, DBMS and its four advantages.
2. Name table / record / field and every key type.
3. Draw ER symbols and 1:1, 1:M, M:N (junction table!).
4. Explain Cascade Update and Cascade Delete.
5. Walk the Library Management ER from requirements to BorrowRecord.
6. Tables store, Forms enter, Queries search, Reports print.
7. Build SELECT / AND / OR / Totals queries and pick the right chart.

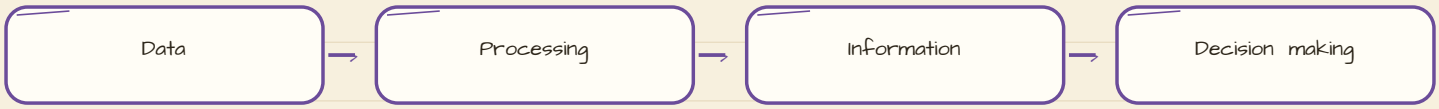
4.1.1 Data, Information and Database

Data

Unprocessed facts, figures, or observations. Quantitative (80, 75, 90) or qualitative (colour of cars). Less useful alone.

Information

Organised, processed, interpreted data that supports decisions. Example: class average is 80%.



Aspect	Data	Information
Definition	Raw facts and figures	Processed and meaningful data
Example	85, 90, 75	Average marks = 83.3%
Usefulness	Less useful on its own	Useful for decision making

A database is an organised collection of related data stored so it can be easily accessed, managed, and updated. Traditional files/books made searching hard - a database solves that.

4.1.2 Database Management System (DBMS)

Software that lets users create, manage, and interact with databases. It is the bridge between the user and the data (MySQL, Oracle, Access). Users do not handle raw files.

1. Less redundancy

Same data is not stored many times. Department name stored once, not in every student row.

2. Improved consistency

Everyone sees the same correct data. Update a phone number once - it updates everywhere.

3. Security / authorised access

Only authorised users see sensitive info (admin sees marks; staff sees names).

4. Integrity / accuracy

Stored data stays reliable - e.g. no duplicate Roll Numbers.

4.2 Database Components

The most basic structure is a table - rows and columns, like a spreadsheet, one entity per table (Students, Teachers).

Component	Description
Table	Rows + columns storing one entity.
Record / Row	Complete related data about ONE instance (Ali's whole row).
Field / Column	One attribute (the entire Names column, or only Marks).

Roll No	Name	Course	Grade
101	Ali	CS	A
102	Sara	IT	B+
103	Ahmed	CS	A

One horizontal line = a Record. One vertical line = a Field.

4.3.1 Keys and Integrity Constraints

* GOLDEN TOPIC

A key field uniquely identifies records and links tables. Essential for efficient design.

Candidate key

Any field (or set) that CAN uniquely identify a record. A table may have several; only ONE becomes the primary key. Unique, not NULL. Employee: EmpID, EmpLicense#, EmpPassport are all candidates.

Primary key

The chosen candidate. Unique, not NULL, exactly ONE per table. Example: EmpID.

Alternate key

Candidate keys NOT chosen as PK. Still unique. EmpLicense# and EmpPassport become alternates.

Foreign key

A field in one table that refers to the PK of another - the link. MAY be duplicate (many employees, one DeptID). MAY be NULL in some designs. DeptID in Employee > DeptID in Department.

~ Classroom IDs

Roll Number, B-Form, Phone are all candidate keys. The school picks Roll Number as the Primary Key.

4.3 Relational Database Management System (RDBMS)

An RDBMS stores data in multiple tables and uses keys to keep relationships. Banks and universities use them.

- Reduces data redundancy.
- Supports secure multi-user access.
- Easy to query with SQL (Structured Query Language).

4.4 Entity Relationship Model (ER-Model)

* GOLDEN TOPIC

A conceptual blueprint used before the database is built - the architect's drawing.



Component	What it is	Symbol
1. Entities	Real-world object/concept we store (Student, Teacher, Course).	Rectangle
2. Attributes	Properties (StudentID, Name, Class). PK attributes are underlined.	Oval
3. Relationships	How entities connect (Student Enrolls in Course).	Diamond

Types of relationships

One-to-One (1:1)

One record - only one record. Rare; often for security. One Student has One ID Card.

One-to-Many (1:M)

One in A - many in B. Most common. One Teacher teaches Many Students.

Many-to-Many (M:N)

Many in A - many in B. Many Students enroll in Many Classes.

> ! **The M:N database rule**
>
> Relational databases cannot store M:N directly - it duplicates data. Break it into two 1:M relationships with a Junction Table (e.g. Enrollment).

4.5 Referential Integrity

* GOLDEN TOPIC

A Foreign Key must always refer to an existing, valid Primary Key in the parent table.

- You cannot enter Result for StudentID 999 if 999 is not in Student (no orphan records).
- You cannot delete a student who still has marks, unless related rows are handled first.

Cascade Update

Change the PK in the parent > the same change is applied in all child tables. Ali 101>201 updates Enrollments automatically. No broken links.

Cascade Delete

Delete the parent > related child rows are deleted too. Sara (102) leaves > her Enrollments and Results vanish. Prevents ghost data.

- Maintains complete data consistency.
- Saves admin time.
- Reduces errors and orphans.
- Reliable reporting.

4.6 Relational Schema Development

A formal blueprint of the database before MS Access (or any RDBMS) is opened.

- How data is organised into tables.
- Which fields each table has.
- Which fields are PK / FK.
- How tables relate.

1. Reduces redundancy.
2. Improves integrity by planning keys first.
3. Makes the database easier to manage and scale.

> * *Teacher's tip*
> Skipping schema design is building a house without a drawing - messy, broken structures.

4.7 Case Study - Library Management System

Step 1 Requirement analysis

- A book has an ID, title, author, publisher.
- A member (student/teacher) has an ID, name, contact.
- A member can borrow multiple books; a book can be borrowed by different members over time.
- Record Issue Date and Return Date.

Steps 2-3 Entities and attributes

Book

BookID (PK), Title, Author, Publisher

Member

MemberID (PK), Name, ContactInfo

Steps 4-5 Relationship + cardinality

A Member borrows a Book. One member > many books over time. One book > many members over time. Conclusion: M:N.

Step 6 Resolve M:N with junction entity BorrowRecord

Each borrowing is a separate row. M:N becomes two 1:M links.

BorrowRecord field	Role
BorrowID (PK)	Unique ID for this transaction
MemberID (FK)	Who borrowed

BorrowRecord field	Role
BookID (FK)	Which book
IssueDate	Standard attribute
ReturnDate	Standard attribute

Step 7 Final structure

Member (1) -- (M) BorrowRecord (M) -- (1) Book

4.8 Different Database Objects

1. Tables

Where data is actually stored. Foundation - all other objects depend on them.

2. Forms

Friendly UI to enter/update data (text boxes, drop-downs). Reduces entry errors vs raw grids.

3. Queries *

Ask questions: "students above 80%". Filter, sort, calculate - original tables stay intact.

4. Reports

Formatted output for print/share (report cards, fee summary). Read-only - you do not enter data here.

*** Four-word summary**

Tables store. Forms enter. Queries search. Reports print.

4.9 Creation of Tables in MS Access

4.9.1 Plan first

- Purpose?
- Which fields?
- Data types (Text / Number / Date)?
- Which field is the PK?

4.9.2 Datasheet View - fast, spreadsheet-like, limited

1. Create tab > Table.
2. Blank grid appears.
3. Rename Field1, Field2 by double-click.
4. Type data; Access guesses types.

! Limitation

Hard to set strict rules ("Marks cannot exceed 100"). Professionals prefer Design View.

4.9.3 Design View - the professional way

1. Create tab > Table Design.
2. Enter Field Name (StudentID, Name).
3. Pick Data Type from the dropdown.
4. Right-click the unique field > Primary Key (key icon).
5. Save the table.

4.9.4 Data types

Type	Used for
Short Text	Names, addresses, phone numbers (you do not add phones). Up to 255 chars.
Number	Values used in maths - Age, Marks, Quantity.
Date/Time	DOB, admission, appointments.
Currency	Fees, salary.
AutoNumber	Unique sequential number per row - perfect PK.

4.10 Designing Forms

Forms minimise input errors and enforce validation visually.

4.10.2 Every form MUST have a data source (Table or Query). A form cannot exist without somewhere to save/retrieve.

4.10.3 Form Wizard

1. Create > Form Wizard.
2. Select table (e.g. Student).
3. > moves chosen fields; >> all fields.
4. Layout: Columnar = one record at a time - best for data entry.
5. Name it (Student Registration) > Finish.

* 4.11 Creating Queries in MS Access

* GOLDEN TOPIC

Most powerful Access feature: retrieve, filter, analyse - without changing original tables.

4.11.2 Simple SELECT

1. Create > Query Design.
2. Show Table > add Student.
3. Double-click fields (Name, Department) into the grid.
4. Red Run (!) executes.

4.11.3 Filtering - Criteria row

Single

Type "Computer Science" under Department - only those rows.

AND (same row)

Name = "Ali" AND Department = "Computer Science" - both on the SAME criteria row.

OR (different rows)

Department "IT" on Criteria, "Computer Science" on the or: row.

4.11.4 Summarisation - Totals () button

Function	What it calculates
COUNT()	How many records (students in IT).
SUM()	Total of values (fees collected).
AVG()	Mathematical mean.
MAX() / MIN()	Highest / lowest in the group.

4.11.5 Example - average marks per department

1. Query + Student table.
2. Add Department and Marks.
3. Click Totals ().
4. Department > Group By. Marks > Avg.
5. Run. CS 82.5 IT 76.0

4.11.6 Data visualisation

Bar (horizontal)

Long category names.

Column (vertical)

Side-by-side category compare (2024 / 2025 / 2026).

Pie

Parts of a whole / percentages.

Line

Trends over time (visitors per month).

Chapter wrap - keep these

- Data = raw facts; Information = processed meaning.
- DBMS: less redundancy, consistency, security, integrity.
- Row = record, column = field. PK unique/not null; FK links tables.
- ER: rectangle / oval / diamond. M:N needs a junction table.
- Referential integrity + Cascade Update / Delete keep parents and children honest.
- Access objects: Tables store, Forms enter, Queries search, Reports print.
- Query criteria: same row = AND, extra row = OR. Totals for COUNT SUM AVG MIN MAX.
- Charts: bar/column compare, pie proportions, line trends.

CHAPTER 5

Application and Impacts of Computing

Computing is already in Pakistani schools, hospitals, banks, farms and factories. This chapter names the three engines (AI, IoT, analytics), where IoT lands, how to judge a source, the plus/minus of each field, and assistive tech.

You will sketch

1. Define AI / IoT / Data Analytics and complete the comparison table.
2. List 6 IoT components and the sensor>cloud>actuator loop.
3. Give IoT uses in cities, industry, health, school, home, farms.
4. AI in Pakistani education - uses, benefits, challenges.
5. Primary / secondary / tertiary sources and when each lies.
6. Plus and minus impacts + responsible-use rules.
7. Four assistive categories and careers.

5.1 Introduction to Computing

We use computers, phones, the internet and smart devices everywhere. In Pakistan this is becoming vital in schools, hospitals, banks, farms and industries.

5.1.1 Artificial Intelligence (AI)

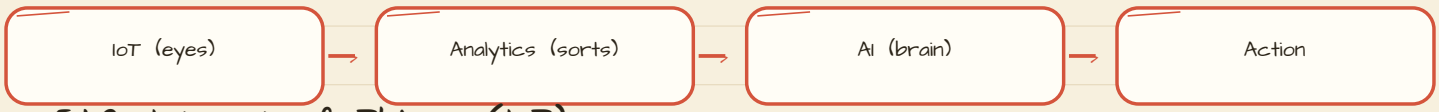
* GOLDEN TOPIC

"Artificial" because humans create it; "intelligence" because it performs tasks that need thinking. AI uses data to decide.

- Phone face recognition to unlock screens.
- YouTube suggesting videos from your interests.
- Chatbots answering questions on a website.

~ *AI is the brain of a smart system*

IoT devices gather data, Analytics sorts it, AI makes the final decision. Room is hot > AI turns the AC on.



5.1.2 Internet of Things (IoT)

* GOLDEN TOPIC

Physical devices connected to the Internet that collect and share data to perform tasks automatically. Example: a smartwatch sends heart-rate data to a phone.

Six foundational components

1. Sensors - eyes & ears

Collect from the physical world: temperature, motion, fingerprint, smoke, soil moisture.

2. Actuators - muscles

Turn digital commands into movement: motor/switch, automatic door, water valve, fan.

3. Connectivity

Without it, sensors cannot talk. Wi-Fi, Bluetooth, ZigBee, 4G/5G.

4. Data processing

Collect, organise, analyse, decide. Is room temp too hot? Trigger AC.

5. Cloud computing

Store/use services over the internet, not a local disk. Google Drive, iCloud, online LMS.

6. User interface

Humans see info and give commands: mobile apps, touch screens, voice.

Sensor collects → Wi-Fi connects → Cloud processes → Actuator acts → User app UI

5.1.3 Data Analytics

Collect, organise, clean, study and present data to find useful information for better decisions. College collects marks, calculates percentages, finds who needs help - that whole pipeline is analytics.

1. Data collection - websites, surveys, sensors, hospital records, bank transactions.
2. Data storage - files, databases, spreadsheets, cloud.
3. Data cleaning - duplicates, typos, missing records, bad formats.
4. Data analysis - patterns and trends (which subjects failed most?).
5. Data visualisation - plots/charts (pie of pass vs fail).

IoT vs AI vs Data Analytics

Aspect	IoT	AI	Data Analytics
Definition	Network of physical devices on the internet.	Machines that think and decide.	Examining data to find patterns.
Main purpose	Data collection and automation.	Intelligent decisions and predictions.	Meaningful insights.
Type of work	Physical devices (hardware).	Smart software systems.	Data (numbers, records).
Output	Raw environmental data.	Smart decisions or actions.	Reports, graphs, insights.

5.2 Uses of IoT in different areas

IoT improves efficiency, saves time, and raises quality of life - modern infrastructure.

1. Smart cities

Traffic signals adapt to sensors. Street lights follow daylight/movement. Parking sensors guide drivers. Smart bins alert when full.

2. Industrial automation

Temperature/vibration sensors prevent breakdowns. Repeatable automated tasks. Quality sensors on the line (Sindh textile).

3. Healthcare

Smartwatches track heart rate. Remote monitoring to doctors (vital for rural Pakistan). Emergency alerts to hospitals.

4. Education

Digital boards + projectors. Biometric attendance. Automated classroom lighting and AC.

5. Home automation

App-controlled lights. Wi-Fi cameras. Voice (Alexa/Google) for TV and appliances.

6. Smart farming (Sindh / Punjab)

Soil-moisture sensors. Pumps ON/OFF from soil data. Drones for crop health. Saves water, raises yield.

Soil sensor → Cloud / AI needs water? → Water pump

5.2.1 AI in education in Pakistan

Practical uses

LMS. Automated grading and digital testing. AI career counselling.

Benefits

Learn at your own speed. Find students who need help. Assistive AI for disabilities.

Challenges

Rural internet. Not enough devices. Teacher training. Data privacy/security.

5.2.2 Career connection

Software houses, telecom (Jazz, Telenor), banks, government IT.

- IoT Engineer - smart systems and connected devices.
- Data Analyst - data from IoT systems.
- Skills: Python / C++, networking (Wi-Fi, protocols), electronics (sensors/circuits), problem-solving.

5.3 Information Sources

* GOLDEN TOPIC

Ways to find knowledge. The internet is vast - not all of it is accurate. Three categories:

Primary (base / firsthand)

Original information from people, events, or official records. Highly authentic. Interviews, photographs, diaries, NADRA official records.

Secondary

Already collected and explained by someone else - summarises or analyses primary data. Textbooks, news reports, research papers, Bureau of Statistics reports.

Tertiary

Highly summarised from primary + secondary. Quick reference. Encyclopedias, Wikipedia, dictionaries, handbooks.

Primary (base data) → Secondary (explained) → Tertiary (quick lists)

When do sources become unreliable?

Source type	Reasons for unreliability
Primary	Incomplete, recorded incorrectly, or deliberately false.
Secondary	Outdated, no citations, or not verified by an expert.
Tertiary	Unknown website, or oversimplified and missing key facts.

* **Rule of thumb**

If an online article has no author, no date, and no references - do not trust it for academic work.

5.4 Impacts of Computing in Various Fields

Life is faster and more connected - but privacy risks and addiction travel with it.

Field	Positive (+)	Negative (−)
Education	Online classes, e-books, apps - flexible access.	Screen time cuts attention; physical health issues.
Communication	Email and video calls - instant, global, cheap.	Less in-person contact; social isolation.
Business	Online shops, digital banking, automation.	Manual jobs lost; cyber-attacks on banks.
Healthcare	Digital records and telemedicine for remote patients.	Patient data at risk if servers are hacked.
Agriculture	Smart irrigation and forecasts raise yields.	Rural internet gaps lock many farmers out.

Smart and responsible use

- Protect personal information; use strong passwords.
- Do not share fake news or unverified forwards.
- Balance screen time with sports and physical activity.

5.5 Assistive Technologies

* GOLDEN TOPIC

Devices, software and systems that help people with disabilities or functional limitations participate equally in school, work and society.

1. Mobility aids

Move from place to place. Smart wheelchairs, stair lifts.

2. Sensory aids

Vision and hearing. Hearing aids, screen readers (VoiceOver), Braille displays.

3. Communication aids

Speech difficulties. Speech-generating devices, text-to-speech apps.

4. Cognitive / learning aids

Memory, reading, understanding. Audiobooks, spell-checkers, digital reminders.

5.5.1 Why they matter

- Equal access to education - screen readers let blind students read digital textbooks.
- Independence - mobility aids reduce constant human help.
- Social inclusion - work, study and socialise with confidence.

5.5.2 Careers

- Assistive software developer - speech-to-text tools.
- Biomedical engineer - artificial limbs, hearing aids.
- UX/UI accessibility expert - sites that work for visually impaired users.

Chapter wrap - keep these

- AI = machines learn, recognise patterns, decide. IoT = connected things. Analytics = insight pipeline.
- IoT parts: sensors, actuators, connectivity, processing, cloud, UI.
- Sources: Primary (original), Secondary (analysed), Tertiary (summaries). Check author, date, references.
- Computing speeds education/business/health/farms but risks privacy, jobs, addiction.
- Assistive tech (mobility, sensory, communication, cognitive) = equality, independence, inclusion.

CHAPTER 6

DIGITAL LITERACY

Digital literacy is not "can you tap an app?" - it is finding, judging, collecting, organising and sharing information safely. The chapter ends with a 12-step inquiry: phones vs study habits.

You will sketch

1. Define digital literacy in the Information Age.
2. Tell qualitative from quantitative and when to use each.
3. Pick interview / survey / prototype / observation / simulation.
4. Contrast primary vs secondary data and design an 8-step collection plan.
5. Match spreadsheet, slides, infographic, report to a purpose.
6. Walk the Digital Inquiry Project (question > artefact) with ethics.

6.1 Introduction to Digital Literacy

The ability to understand and use digital devices, the internet and online tools in a safe, effective and responsible way. Find information, communicate, produce content, solve problems.

> **i** *Did you know?*
>
> Not only using devices - using digital information safely and wisely. Locate it, organise it, turn it into something useful, share it responsibly.

6.2 Digital Literacy in the Information Age

The Information Age: vast data produced, shared and accessed instantly through the internet, mobiles and platforms. Access alone is not enough - students must understand, manage and use information wisely. Digital literacy adds technical, cognitive and social abilities.

★ 6.3 Types of Data

★ GOLDEN TOPIC

6.3.1 Qualitative

What people think, feel, notice, experience. Non-numerical, descriptive. Opinions about online classes, study-habit notes, observation notes.

6.3.2 Quantitative

Numbers. Count, measure, compare, average, chart. Students online daily, test marks, hours online.

Type	Nature	Examples	Best used for
Qualitative	Descriptive, non-numerical	Opinions, reasons, behaviour	Understanding why and how something happens
Quantitative	Numerical, countable or measurable	Hours, scores, percentages, totals	Comparing results and making graphs

★ 6.4 Data-Collection Strategies

★ GOLDEN TOPIC

Digital literacy is not only finding what already exists - students also gather original information.

Interviews → Surveys → Prototypes → Observation → Simulations

6.4.1 Interviews

Planned conversation, one-to-one or small group. Questions planned in advance. Answers usually qualitative.

6.4.2 Surveys / questionnaires

Same questions to many people. Paper or Google/Microsoft Forms. Online tools collect responses and draw basic charts.

6.4.3 Prototypes

Simple early version of a product/model. Users try it and give feedback - data for improvement - before the expensive final build.

6.4.4 Observation

Watch what people do; record carefully. No direct questions. Notes (qual) or counts (quant).

6.4.5 Simulations

Computer model of a real situation. Change variables and watch. Used when real experiments are costly, risky or hard.

* **Which method?**

Cars through an intersection at 8 AM > Observation (count). Feelings about cafeteria food > Survey / Interview.

6.5 Primary and Secondary Data

* GOLDEN TOPIC

Where data came from affects how much control you had and how confidently you can use the results.

6.5.1 Primary

Collected first-hand by the researcher for this purpose - from people, events, situations. Interviews, surveys, observations, experiments. Fresh, you do the work.

6.5.2 Secondary

Already collected by someone else, reused. Books, research articles, official reports, web stats, online databases. Existing archives.

Aspect	Primary	Secondary
Source	Collected directly by the researcher.	Already collected by others.
Examples	Survey responses, interviews, observations.	Reports, articles, websites, databases.
Purpose	Prepared for a specific study.	Reused for a new purpose.
Control	High control over questions and process.	Limited - the data already exists.

6.6 Designing a data-collection approach

Plan first so what you gather is accurate, relevant and useful.

1. Define the research question clearly.
2. Decide the type of data: quantitative, qualitative, or both.
3. Select methods (survey, interview, observation).
4. Choose the sample / target group.
5. Prepare tools: forms, questions, observation sheets.
6. Collect carefully and accurately.
7. Store digitally in a spreadsheet and keep a backup.
8. Ethics: respect privacy. Consent is mandatory.

6.7 Presenting Data Using Digital Tools

Collection is only half. Present findings so others understand them quickly.

6.7.1 Spreadsheets (Excel, Google Sheets, Numbers)

Rows and columns. Enter systematically. Totals, averages, percentages. Sort and filter. Auto charts.

6.7.2 Presentations (PowerPoint, Slides, Canva)

Title + research question. Short bullets, not paragraphs. Tables, charts, key findings. Simple readable design.

6.7.3 Infographics (Canva, Power BI, Tableau)

Visual facts so the main idea is instant. Simplify, highlight findings, limited text, accurate.

6.7.4 Reports (Word, Google Docs)

Title, intro, purpose. Data as tables/charts. Simple-language findings. Conclusions and recommendations. Optional AI wording - always review accuracy.

Spreadsheet data → Chart / graph → Insert into slides

* *Visual rule of thumb*

A good chart explains faster than a long paragraph - but it must be accurate and never exaggerated.

6.8 Case Study - Digital Inquiry Project

* GOLDEN TOPIC

Apply the whole chapter to a real question.

Research question: How does daily mobile phone use affect the study habits of students?

Classmates use phones in free periods and while studying. Some say it helps; others say it wastes time. Investigate.

1 Question → 2 Search → 3 Collect → 4 Analyze → 5 Create

Steps 1-3 Methodology

Step 1 Advanced search

Quotes for exact phrases: "mobile phone use" "study habits" students. filetype:pdf for official docs. site:.edu for educational sites. Checklist: author? recent? .edu/.gov/research? facts or bias?

Step 2 Data types

Quantitative: hours daily, % who feel distracted. Qualitative: why they get distracted, suggestions.

Step 3 Method

Same questions to many students > Survey / questionnaire (Google Forms or paper).

Steps 4-6 Survey, sample, ethics

Question	Type of data
How many hours do you use your phone daily?	Quantitative
Which activity do you use your phone for the most?	Quantitative (categories)
How does your phone help you in studies?	Qualitative
Suggest one way to use phones responsibly.	Qualitative

Step 5 Sample: 40 Class XI students from different sections - a representative mix.

Step 6 Collect ethically:

- Ask permission (consent).
- Avoid sensitive personal information.
- Do not record names unless necessary (anonymity).
- Keep responses private.
- Use data only for education.

Steps 7-9 Secondary data, spreadsheet, charts

Step 7: include a reliable article - "excessive screen time lowers concentration" - and compare it with your survey.

Student	Hours	Study use	Main activity	Distracted?
1	3	Yes	Lectures	Sometimes
2	5	No	Social media	Yes
3	2	Yes	Notes	No

Step 9: Daily phone-use chart - bins 1-3 hrs / 3-5 hrs / 5+ hrs.

Steps 10-12 Analyse, conclude, recommend

- Most students use phones 3-5 hours daily.
- Half use them for lectures/notes.
- Entertainment-heavy users report much higher distraction than study users.

* Conclusion (keep this wording)

Mobile phones have both positive and negative effects. They provide learning materials. Excessive social media and gaming reduce concentration. Therefore phones should not be banned, but managed wisely.

1. Turn off social-media notifications while studying.
2. Set daily screen-time limits.
3. Take physical breaks from screens.

The final digital artefact

A product made with digital tools that answers the research question with evidence - not just a decorated file.

Artefact	Tools	Purpose
Digital report	MS Word, Google Docs	Full structured write-up.
Presentation	PowerPoint, Canva, Slides	Key findings for a live audience.
Infographic	Canva, Adobe Express	Main results on one visual page.
Dashboard	Power BI, Tableau	Interactive charts and filters.

1. Title
2. Research question
3. Introduction
4. Method of data collection
5. Primary data findings
6. Secondary data summary
7. Charts and graphs
8. Conclusion
9. Recommendations
10. References

Careers in digital research

Market research analyst

Designs surveys (primary data) to guide business.

UX researcher

Qual + quant to improve apps and sites.

Data journalist

Advanced search + infographics for stories.

Digital marketing specialist

Audience data via dashboards and reports.

Chapter wrap - keep these

- Digital literacy = find, evaluate, create, share - safely and wisely.
- Qualitative = words/why. Quantitative = numbers/charts.
- Primary = you collect it. Secondary = someone already did.
- Methods: interview, survey, prototype, observation, simulation.
- Advanced search: "quotes", filetype:pdf, site:.edu.
- Ethics: consent + privacy.
- Tools: sheets (organise), slides (deliver), infographics (glance), reports (depth).
- Artefact = evidence-backed answer to the research question.